DAY 2 | HTTP SERVERS | REST APIS

Node.js: Everything You Need to Know

Day 2: HTTP Servers, REST APIs & Event-Driven Architecture

Build a real network service from scratch. We'll cover the HTTP request/response cycle, manual routing, body parsing, and the event-driven concurrency model that makes Node.js powerful.

By **Wesam Abousaid**

LECTURE PLAN

NodeJS — Day 2



Client-Server Model

15 min



HTTP Fundamentals

30 min



HTTP Server Deep Dive

45 min



Routing & Query Params

45 min



Request Body & Helpers

45 min



Event-Driven Concurrency

30 min

Project Goal: Transform the CLI task manager from Day 1 into a network service exposing a REST API.

Learning Objectives

By the end of Day 2, students should be able to:

- [ ] **Client-Server Model** — Explain HTTP request/response flow.
- [ ] **HTTP Server** — Build one using Node's built-in http module.
- [ ] **Routing** — Handle query parameters and dynamic paths.
- [ ] **Body Parsing** — Safely parse request bodies from streams.
- [ ] **REST API** — Connect endpoints to file storage.
- [ ] **Concurrency** — Understand non-blocking I/O in Node.

Success criteria: Students can run the server, interact via browser or Postman, and handle multiple simultaneous requests asynchronously.

15 MIN

Client-Server Model

The foundation of all web communication — understanding how browsers and servers talk to each other.

Client

(Browser)

Server

(Node.js



HTTP Request Example:

```
GET /tasks HTTP/1.1
Host: localhost:3000
Accept: application/json
```

HTTP Response Example:

```
HTTP/1.1 200 OK
Content-Type: application/json

[{"id":1,"title":"Buy milk","done":false}]
```

Interactive: Ask students — 'What happens if two users visit at the same time?' → Leads to event-driven, non-blocking discussion.

30 MIN

HTTP Fundamentals

Understanding the building blocks of every web request and response.

GET

Retrieve data

POST

Send new data

PUT

Update existing data

DELETE

Remove data

Status Codes

200 OK	201 Created
400 Bad Request	404 Not Found
500 Server Error	

Headers

- Content-Type
- Authorization
- Accept

Request Body

Typically JSON for POST/PUT requests.

Code Demo – Minimal Server:

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello from Node server!");
});

server.listen(3000, () => console.log("Server running at http://localhost:3000"));
```

45 MIN

Node HTTP Server Deep Dive

Inspecting the request object and building structured JSON responses with manual routing.

req.method

Identifies the HTTP verb: GET, POST, DELETE, etc.

req.url

The full path and query string of the incoming request

req.headers

Metadata sent with the request: Content-Type, Authorization, Accept

Code Example – JSON Routing:

```
const tasks = [];

http.createServer((req, res) => {
  if (req.method === "GET" && req.url === "/tasks") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify(tasks, null, 2));
  } else {
    res.writeHead(404);
    res.end("Not Found");
  }
}).listen(3000);
```

45 MIN

Routing, Query Params & Dynamic Paths

Parsing URLs, filtering with query parameters, and handling dynamic route segments.

url.parse()

Breaks the URL into pathname and query: url.parse('/tasks?done=true', true)

Query Params

Filter data without a new route: GET /tasks?done=true → query.done === 'true'

Dynamic Paths

Extract IDs from the URL: /tasks/42 → pathname.split('/')[2] === '42'

Method Routing

Same path, different behavior: GET /tasks lists, DELETE /tasks/42 removes

Code Example – Full Routing Logic:

```
const url = require("url");

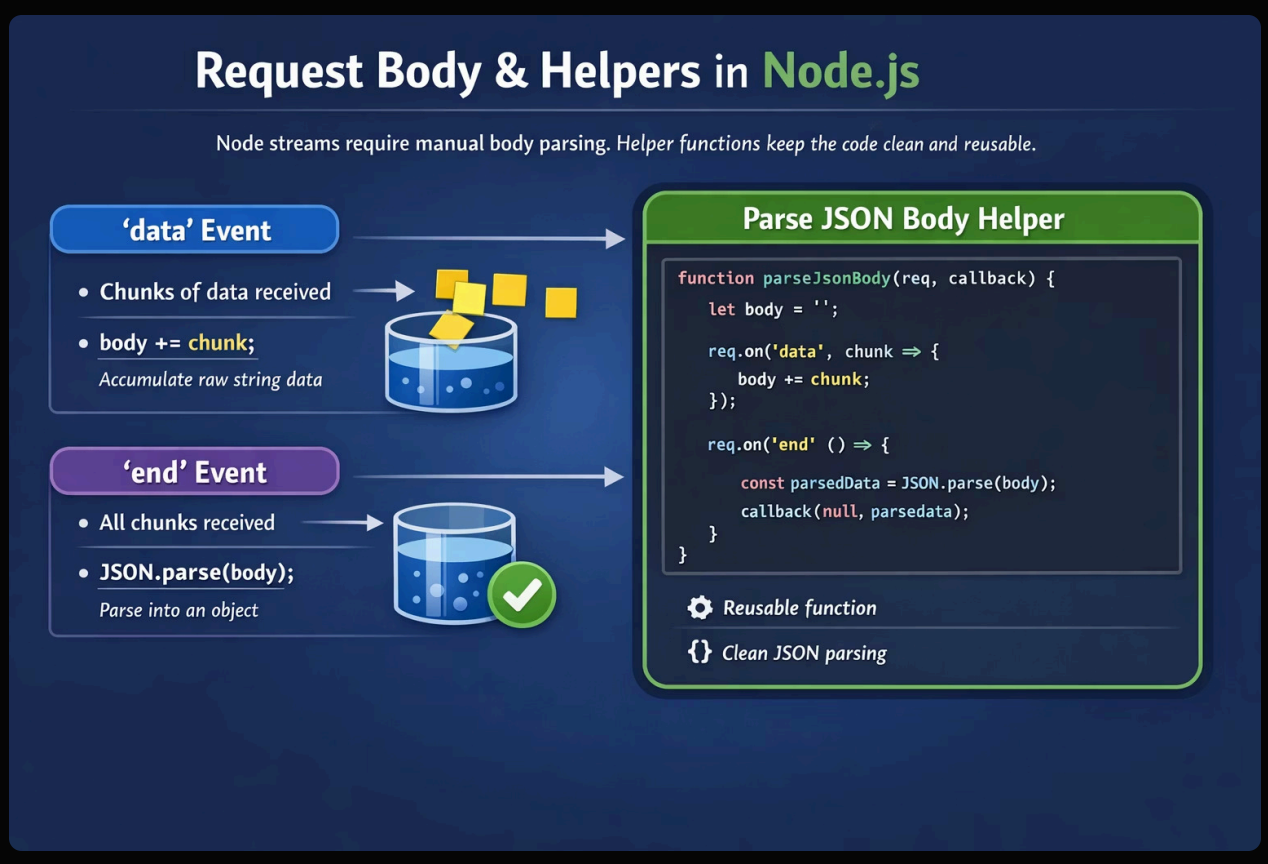
http.createServer((req, res) => {
  const parsed = url.parse(req.url, true);
  const pathname = parsed.pathname;
  const query = parsed.query;

  if (pathname === "/tasks" && query.done) {
    res.end(JSON.stringify(tasks.filter(t => t.done === (query.done === "true"))));
  } else if (pathname === "/tasks") {
    res.end(JSON.stringify(tasks));
  } else if (pathname.startsWith("/tasks/") && req.method === "DELETE") {
    const id = Number(pathname.split("/")[2]);
    const index = tasks.findIndex(t => t.id === id);
    if (index === -1) return res.end(JSON.stringify({ error: "Task not found" }));
    tasks.splice(index, 1);
    res.end(JSON.stringify({ message: "Task deleted" }));
  } else {
    res.writeHead(404);
    res.end("Not Found");
  }
}).listen(3000);
```

45 MIN

Request Body & Helpers

Node streams require manual body parsing. Helper functions keep the code clean and reusable.



Parse JSON Body Helper:

```
function parseJSONBody(req, callback) {
  let body = "";
  req.on("data", chunk => body += chunk);
  req.on("end", () => {
    try {
      callback(null, JSON.parse(body));
    } catch (err) {
      callback(err);
    }
  });
}
```

Structured Response Helper:

```
function sendJSON(res, status, payload) {
  res.writeHead(status, { "Content-Type": "application/json" });
  res.end(JSON.stringify(payload, null, 2));
}
```

Example – POST /tasks:

```
if (req.method === "POST" && req.url === "/tasks") {
  parseJSONBody(req, (err, newTask) => {
    if (err) return sendJSON(res, 400, { error: "Invalid JSON" });
    newTask.id = Date.now();
    newTask.done = false;
    tasks.push(newTask);
    sendJSON(res, 201, newTask);
  });
}
```

30 MIN

Event-Driven Concurrency

Node.js handles thousands of concurrent requests using a single thread and non-blocking I/O — no waiting, no blocking.

Single-Threaded

One thread handles all requests — no context switching

Non-Blocking I/O

File and network ops don't pause execution

Callback Queue

Async callbacks wait until I/O completes

Event Loop

Continuously processes the callback queue

Demonstration – Async File Reads:

```
const fs = require("fs");

fs.readFile("tasks.json", (err, data) => {
  if (err) console.error("File read failed");
  console.log("Read complete"); // Logs may appear out of order
});
```

Note: Logs may appear out of order — this is expected and intentional.

LAB 1

Lab 1 – Hello Server

Build your first Node HTTP server and verify it responds in the browser.

Goal:

Create server.js that responds with plain text or JSON when visited in a browser.

01

Create server.js

Create a new file in your project directory

02

Require http

const http = require('http')

03

Create the server

Respond with JSON: { message: 'Hello from Node!' }

04

Listen on port 3000

server.listen(3000, callback)

05

Verify in browser

Open http://localhost:3000

Starter Code:

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "application/json" });
  res.end(JSON.stringify({ message: "Hello from Node!" }));
});

server.listen(3000, () => console.log("Server running at http://localhost:3000"));
```

LAB 2

Lab 2 – GET /tasks

Serve a list of tasks from memory or a tasks.json file over HTTP.

Goal:

Extend server.js to handle GET /tasks and return a JSON array of tasks.

01

Define Tasks Array

Create in memory or load from tasks.json

02

Add GET Route

req.method === 'GET' && req.url === '/tasks'

03

Test the Endpoint

Use curl or open in browser

Test Command:

```
curl http://localhost:3000/tasks
```

Expected Response:

```
[
  { "id": 1, "title": "Buy milk", "done": false }
]
```

LAB 3

Lab 3 – POST /tasks

Accept new tasks from the client, assign them an ID, and persist them to tasks.json.

Goal:

Handle POST /tasks — parse the request body, add an ID and done:false, push to the array, and write back to tasks.json.

01

Add parseJSONBody

Helper to collect stream chunks and parse JSON

02

Add sendJSON

Helper to write status, Content-Type, and payload

03

Handle POST /tasks

Check req.method === 'POST' && req.url === '/tasks'

04

Assign ID & done

newTask.id = Date.now(); newTask.done = false

05

Persist to File

Write updated array to tasks.json with fs.writeFile

06

Respond 201

sendJSON(res, 201, newTask)

Test Command:

```
curl -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Buy milk"}'
```

Expected Response:

```
{ "id": 1712345678901, "title": "Buy milk", "done": false }
```

LAB 4

Lab 4 – DELETE /tasks/:id

Remove a task by its ID from the array and persist the change to tasks.json.

Goal:

Handle DELETE /tasks/:id — extract the ID from the URL, find the task, remove it, and respond with a confirmation or error.

01

Check Route

pathname.startsWith('/tasks/') && req.method === 'DELETE'

02

Extract ID

const id = Number(pathname.split('/')[2])

03

Find Task

tasks.findIndex(t => t.id === id)

04

Handle 404

If index === -1, respond with { error: 'Task not found' }

05

Remove Task

tasks.splice(index, 1)

06

Persist to File

Write updated array to tasks.json

07

Respond Success

sendJSON(res, 200, { message: 'Task deleted' })

Test Command:

```
curl -X DELETE http://localhost:3000/tasks/1712345678901
```

Error Case:

```
{ "error": "Task not found" }
```

OPTIONAL

Lab 5 – Optional Challenge: Filtering

Add query parameter support to filter tasks dynamically without changing the route.

Goal:

Support GET /tasks?done=true and GET /tasks?done=false to return only matching tasks.

01

Parse the URL

url.parse(req.url, true) extracts the query object

02

Check query.done

Verify the parameter exists before filtering

03

Filter Tasks

tasks.filter(t => t.done === (query.done === 'true'))

04

Return JSON

sendJSON(res, 200, filtered)

Code Snippet:

```
const parsed = url.parse(req.url, true);
const query = parsed.query;

if (parsed.pathname === "/tasks" && query.done !== undefined) {
  const filtered = tasks.filter(t => t.done === (query.done === "true"));
  return sendJSON(res, 200, filtered);
}
```

Test Commands:

```
curl http://localhost:3000/tasks?done=true
curl http://localhost:3000/tasks?done=false
```

Common Student Mistakes

Watch out for these frequent errors when building HTTP servers in Node.js.



Ignoring Request Method

Always check both the HTTP method and the path.



Reading Body Too Early

Wait for the request body to finish streaming before parsing it.



Blocking File Operations

Use asynchronous file reads so the server stays responsive.

Expected Outcomes – Day 2

By the end of Day 2, students will have built a fully functional REST API from scratch using only Node's built-in modules.

- [ ] **REST API**
  GET, POST, and DELETE endpoints fully working
- [ ] **Query Filtering**
  Dynamic task filtering via ?done=true / ?done=false
- [ ] **Error Handling**
  Proper HTTP status codes: 400, 404, 500
- [ ] **Non-Blocking I/O**
  All file operations use async fs methods
- [ ] **Day 3 Ready**
  Clean codebase prepared for modularization

Students leave with a running server they built themselves — no frameworks, no magic.